

LLM-Driven TLA+ Specification of Essential Paxos

Ana Mako (6418643)
TU Delft

Georgios Markozanis (6551254)
TU Delft

Vuk Jurišić (5513847)
TU Delft

1 Introduction

Consensus protocols are well known for being difficult to understand, specify, and verify correctly [1]. Even implementations that appear correct during testing may fail under rare execution schedules that are difficult to anticipate and reproduce. Because safety properties such as agreement must hold for every possible execution, testing alone provides limited assurance of correctness.

Formal specification and model checking offer a stronger alternative. TLA+ [2] provides a mathematical language for describing distributed systems, and given a specification the TLC model checker can exhaustively explore all reachable states within a finite configuration to verify whether desired properties hold [3]. This is particularly attractive for consensus protocols, whose correctness relies on invariants that must hold universally rather than for a sampled subset of executions. Writing a correct specification, however, remains difficult [2]. The developer must translate an informal protocol description into a precise state-machine model while satisfying TLA+'s syntactic and semantic requirements, which largely confines formal specification to experts in both distributed systems and formal methods.

Recent advances in Large Language Models (LLMs) raise the possibility of lowering this barrier. Modern LLMs have demonstrated strong performance on code-generation and program-synthesis tasks, suggesting that they may also assist in translating protocol descriptions into formally verifiable specifications [4]. Whether such models can reliably generate correct TLA+ specifications, however, remains an open question that is directly testable using SysMoBench [5], a benchmark for evaluating AI-generated TLA+ specifications. SysMoBench assesses generated specifications across four dimensions: syntactic correctness, successful model checking, conformance to execution traces, and satisfaction of expert-written safety invariants.

As a case study, we consider Essential Paxos [6], the minimal core implementation of Paxos contained in the cocagne/paxos library. Unlike the library's extended variants, which introduce additional mechanisms such as leader election, negative acknowledgements, and durable storage, Essential Paxos implements only the core consensus protocol. Its compact size and well-understood safety properties make it an attractive target for formal specification and verification.

Using Essential Paxos, we extend SysMoBench with a new benchmark task consisting of a custom instrumentation harness, execution traces, and expert-written safety invariants. We then use this benchmark to evaluate two LLM-based specification-generation strategies: a single-pass prompting approach and an iterative approach that incorporates feedback from failed verification attempts.

Research Question. *Does feedback-driven prompt refinement improve the quality of LLM-generated TLA+ specifications, and are the resulting prompts transferable across Claude and GPT?*

To answer this question, we conduct three experiments. Experiment 1 studies the iterative refinement process itself, recording how specifications evolve as verification feedback is incorporated into successive prompts. Experiment 2 compares the final prompts produced by each strategy across both models in clean sessions, allowing us to isolate the effect of prompt design from conversational history. Experiment 3 evaluates successful model-prompt combination under increasingly complex protocol configurations to understand how specification quality and verification cost scale with problem size. Our code and artifacts can be found at <https://github.com/makhana/SysMoBench-research>.

2 Formal Modeling

2.1 Protocol Overview

Essential Paxos [6] implements single-decree consensus over three roles, *proposers* initiate ballots, *acceptors* vote, and *learners* detect a converged quorum, in two phases. In Phase 1 a proposer broadcasts `PREPARE( $n$ )` with a fresh ballot n . Each acceptor that has not promised a higher ballot replies `PROMISE( $n, v_a, n_a$ )` carrying its highest previously-accepted value v_a and ID n_a . In Phase 2, on a quorum of promises the proposer issues `ACCEPT( $n, v$ )`, reusing the highest previously-accepted value if any, otherwise its own and acceptors that have not since promoted record (n, v) and reply `ACCEPTED`. A learner declares consensus once a quorum sends `ACCEPTED` for the same ballot.

Scope restriction. The cocagne/paxos library provides four further modules: `practical.py` (NACKs, leadership callbacks, multi-instance resolution), `durable.py` (on-disk persistence), `functional.py` (a stateless wrapper), and `external.py` (custom transport). Our model captures only `essential.py` (three role types, two phases, and simple majority quorums).

2.2 TLA+ Specification Structure

The generated `EssentialPaxos` module declares the constants `Acceptors`, `Proposers`, `Learners`, `Values`, `NoValue`, and `MaxBallot`, with the proposal-ID space defined as `ProposalIDs == (0..MaxBallot) \X Proposers`. State is held in flat, per-role variables (proposer ballot/value/promise-tracking, acceptor promised/accepted IDs and values, learner quorum bookkeeping) plus a global message bag `msgs`. The flat layout is itself a prompt constraint, since nested record types triggered SANY parse errors. The next predicate is the disjunction of five actions corresponding one-to-one to the Python handlers: `Prepare`, `HandlePrepare`, `HandlePromise`, `HandleAccept`, and `HandleAccepted`.

2.3 Software Analysis Technique: TLA+ Model Checking with SysMoBench

Algorithm. The verification pipeline follows four sequentially dependent stages drawn from the SysMoBench benchmark [5]:

- (1) **Syntax correctness.** SANY statically parses and type-checks the generated `.tla` module, catching lexical errors, undefined operators, and arity mismatches.
- (2) **Runtime correctness.** TLC enumerates the reachable state space of `Spec == Init /\ [][Next]_vars` under the `.cfg` configuration, verifying termination and declared invariants, a proxy for logical self-consistency.
- (3) **Conformance.** Each Python execution trace is cut into (pre-state, post-state) windows. TLC checks whether the corresponding action permits each observed transition. The TV score is the fraction accepted.
- (4) **Invariant correctness.** The four expert invariants are concretized against the generated spec’s variable names and checked as `INVARIANT/PROPERTY` clauses.

The canonical aggregate score weights these stages 0.15, 0.15, 0.35, and 0.35 respectively [5]. The heavy weight on the final two stages reflects that syntax and basic runtime correctness are necessary but insufficient conditions for a *faithful* specification.

Implementation architecture. SysMoBench is implemented as a Python package with a CLI entry point (`sysmobench`). Each task is declared by a `task.yaml` descriptor that names the source repository, source files, `specModule`, trace folder, and TV configuration (target actions, harness type, and a label-to-action map). The `direct_call` method submits a single structured prompt to the configured model and writes the generated `.tla` and `.cfg` files to a timestamped output directory. Subsequent stages are invoked sequentially, each receiving the output of the previous stage. A stage failure causes all later stages to be skipped, treating its score as zero in the aggregate.

The transition-validation stage is the most architecturally complex. Agents read the `tv-eval SKILL.md` guide, inspect the spec, and issue TLC queries for each trace window. This agentic loop is non-deterministic and may take 15–60 minutes per (task, model) cell, incurring approximately \$1–4 in API cost.

Implementation limitations. The SysMoBench pipeline has several important limitations.

- **Agent-specific TV runner.** The transition-validation scripts configured for the Claude Code agent. Any other agent must run the stage manually via the skill, deviating from the documented README procedure.
- **Trace coverage.** The harness runs four deterministic scenarios (happy path, dueling proposers, message loss, late message). Real deployments exercise a far richer set of concurrent-proposer races. The TV score reflects conformance to this narrow scenario set, not to all possible executions.
- **Agentic TV non-determinism.** The transition-validation step is itself performed by an LLM agent. Variability in the agent’s variable-name translation can cause individual windows to be misclassified even when the underlying spec is correct, artificially lowering the TV score.

Threats to validity. Three threats apply to our study.

- **Internal validity.** Iterative prompt refinement introduces a feedback loop. Each revision is informed by a specific

SANY/TLC error, so the final prompt is overfitted to the errors encountered in our particular execution environment (Java version, TLC release, model configuration). A different environment might expose different errors and require different prompt constraints.

- **Construct validity.** The SysMoBench aggregate score measures only the four mechanically-checkable dimensions. A specification could score 100% while still being unsound in ways that fall outside the invariant template (e.g., modeling a subtle over-approximation that happens never to fire on the trace set).
- **External validity.** Essential Paxos is a single, educational protocol implementation scoped to 202 lines. Results on prompt engineering strategies may not generalize to more complex industrial protocols (e.g., Raft, CURP) that have larger source files, more message types, and richer invariant sets.

2.4 Prompt Design and Iterative Refinement

We used the SysMoBench `direct_call` method, which submits a single structured prompt containing a task description, a scope section, a network model description, output format requirements, and the full reference Python implementation (via the `{source_code}` template placeholder). A second prompt (`phase2_config.txt`) instructs the model to generate the companion `.cfg` file, and a third (`phase3_invariant_implementation.txt`) produces the `EssentialPaxosInvariants` module that concretizes the expert-written invariant templates against the generated spec’s variable names. We have 2 final versions of all 3 prompts that were created by letting GPT and Claude agents iterate from codebase examples in order to get a TLA specification that passes as many steps as possible. Results are shown and discussed in Section 3.

2.5 Evaluation Setup and Reproducibility

All experiments ran on macOS 26.5.1 with Python 3.11.13, OpenJDK 23.0.1, and TLC 1.8.0. We used SysMoBench’s four-stage evaluation pipeline. Claude Sonnet 4.5 and GPT-5 generated specifications while Claude Code and Codex supported refinement and transition validation. The dataset was `paxos/essential.py` from `cocagne/-paxos` (commit `cf3b5a2`). The scope was the 202-line Essential Paxos core. Essential Paxos is absent from SysMoBench 1.0 so we therefore added it as a new task in `task.yaml`. The task includes prompts, invariants, and a trace harness. The harness leaves upstream source code unchanged using sentinel initialization, external messaging, and traced execution. A `TracedLearner` wrapper avoids Python-3 None comparisons. Protocol ordering semantics are otherwise preserved. The artifact repository includes the task descriptor, prompts, harness, invariant templates, generated specifications, logs, and README commands needed to reproduce the reported results.

3 Correctness Specifications and Verification Results

This section evaluates the generated TLA+ specifications across the four SysMoBench stages: syntax checking, runtime model checking, transition validation, and invariant verification. After defining

the correctness properties and TLC configuration, we report three experiments: iterative prompt refinement (3.3), cross-model prompt transfer (3.4), and configuration scaling (3.5).

3.1 Correctness Properties

We evaluate four expert-written invariants provided by SysMoBench:

- **Agreement:** at most one value is ever decided across all learners at all times.
- **Validity:** any decided value was proposed by some proposer, consensus cannot fabricate values.
- **Stability:** once an acceptor accepts value v at proposal ID n , it never accepts a different value at the same n .
- **PromiseMonotonic:** an acceptor’s promised proposal ID is non-decreasing over time.

3.2 Model Checking Parameters

TLC is configured with: Acceptors = {a1, a2, a3}, Proposers = {p1, p2}, Learners = {l1}, Values = {v1, v2}, MaxBallot = 3. The constant NoValue is represented as a TLC model value acting as an opaque sentinel. Deadlock checking was disabled (CHECK_DEADLOCK FALSE), since Paxos can terminate normally after reaching consensus. The temporal property PromiseMonotonic was specified using PROPERTY rather than INVARIANT. This is because it constrains transitions between states instead of describing a single-state condition.

3.3 Experiment 1: Self-Iteration Phase

In this experiment each model refines its own prompt independently, starting from an initial prompt the model drafts after inspecting the existing SysMoBench prompts and the Essential Paxos repository. Each SANY/TLC failure is fed back automatically and the model regenerates until all four stages pass. We record what errors each model makes and in what order.

3.3.1 Initial Prompt. We gave this initial prompt to both LLMs used in order to acquire results of experiment 1:

“We are researching LLM capabilities of generating TLA specs autonomously using SysMoBench as evaluation tool. Your task is to generate a prompt that fed into an LLM makes LLM produce the TLA specification for Essential Paxos. At the same time use provided SysMoBench repo skills and generate a TLA spec yourself while editing prompts that passes all 4 stages and take notes of SysMoBench failures and prompt changes that made it work in the next iteration. Here is EssentialPaxos repository for reference github_link”

3.3.2 Claude Sonnet 4.5 results. Table 1 summarizes the eight generation attempts and the root cause of each failure prior to the final passing run. The trajectory moved from configuration and state-space problems (unassigned learners, an unbounded CHOOSE $n \in \text{Nat}$, an infinite state space), through a type error on opaque model values, then two SANY parse errors, and finally a TLC runtime exception, before all four stages passed on run 8.

The most impactful constraints the agent converged on were:

- (1) **Bound the proposal ID space.** Replacing $\text{Nat} \setminus \text{X Proposers}$ with $(0.. \text{MaxBallot}) \setminus \text{X Proposers}$ eliminates the infinite state space that causes TLC to diverge.

Table 1: Prompt refinement iteration history (Claude Sonnet 4.5)

Run	Outcome	Root cause
1	FAIL	Learners not assigned in .cfg
2	FAIL	Unbounded CHOOSE $n \in \text{Nat}$
3	FAIL	$>$ applied to model value
4	FAIL	Infinite state space
5	FAIL	SANY parse error
6	FAIL	SANY parse error
7	FAIL	TLC exception
8	PASS	All four metrics passing

Table 2: Prompt refinement iteration history (GPT-5)

Run	Outcome	Root cause
1	FAIL	TLA+ did not compile
2	PARTIAL	TV/invariants failed
3	PARTIAL	Malformed invariants
4	PASS	All four metrics passing

- (2) **Ballot-only comparison.** TLC model values (opaque identifiers) do not support the $<$ operator. Adding $\text{ProposalID} \leq t(a, b) == a[1] < b[1]$ and forbidding $a[2] < b[2]$ eliminates runtime type errors.
- (3) **Flat variable declarations.** SANY rejects multi-line nested record types used to group per-role variables. Requiring one variable per line avoids this class of parse errors.
- (4) **Inline quorum check.** Replacing $\text{CHOOSE } n \in \text{Nat} : \forall E \ Q \dots$ with $\text{Cardinality}(S) * 2 > \text{Cardinality}(\text{Acceptors})$ is necessary because TLC cannot enumerate infinite sets.

The final specification reaches 78.7% TV with all four invariants passing (Table 3)

3.3.3 GPT-5 results. Table 2 summarizes the four refinement iterations required before all SysMoBench stages fully passed.

The first generated specifications failed to compile because the prompt allowed several TLC-incompatible constructs, including unbounded ballot domains, tuple/model-value comparisons, and cfg bindings for tuple sentinels. These issues were resolved by introducing finite proposal-ID spaces, numeric proposer identifiers, explicit ordering helpers, and a ban on comparisons involving opaque TLC model values.

After compilation succeeded, the remaining failures were primarily caused by mismatches between the generated state abstraction and the execution traces used during transition validation. The most impactful constraints that emerged were:

- (1) **Move value selection into Prepare.** The Python helper `set_proposal(value)` is not explicitly traced by the harness. Folding value selection into the Prepare action aligned the generated transitions with the observed execution traces.
- (2) **Derive learner state from msgs.** Learner quorum detection was rewritten to derive accepted messages directly

from the global message set rather than hidden learner bookkeeping variables. This allowed non-decision deliveries to appear as valid steps during transition validation.

- (3) **Restrict invariant syntax to TLC-safe formulas.** The initial invariant translation generated malformed temporal formulas and invalid filtered set comprehensions. The prompt was therefore constrained to emit `PromiseMonotonic` as a single `[] [. . . ]_vars` property and to express `Validity` using simple set membership.

The final specification passes all four stages at 100% TV (Table 3).

3.3.4 Comparison of Iteration Trajectories. Claude required 8 iterations before all SysMoBench stages passed while GPT required 4. Both models independently encountered the same class of finite-domain errors. Both needed explicit `ProposalIDLT` helpers, bounded ballot sets, and majority checks expressed as cardinality inequalities over finite sets. Claude was uniquely affected by parse errors from multi-line nested record types and by the SANY forward-reference restriction, which caused operators referencing variables to be rejected when declared before the `VARIABLES` block. GPT was uniquely affected by message union helpers being wrongly counted as protocol actions (causing action-decomposition partial failures), and by temporal formula shape errors in the `PromiseMonotonic` invariant (`[] [] [. . . ]_vars` instead of the correct `[] [. . . ]_vars`).

The final prompts also differ in their learner-quorum helper. `PromptX` derives `LatestAcceptedBy` from `msgs`, which is why it reaches 100% TV against `PromptC`’s 78.7%.

3.4 Experiment 2: Comparing Prompts Across Models

Experiment 2 evaluates whether the refined prompts from Experiment 1 transfer across models. We take the final Claude Sonnet (`PromptC`) and GPT-5 (`PromptX`) prompt, run each in a fresh session on both models, and evaluate the resulting specifications using the full SysMoBench pipeline. Table 3 summarizes the results.

The outcome is strongly model-specific. Both models pass when paired with their own refined prompts, while both cross-model combinations fail during compilation. With `PromptX`, Claude emitted the `NextProposalNum` helper before the `VARIABLES` block, which SANY rejects as a forward reference, `PromptX` implicitly relied on GPT-5’s generation order. With `PromptC`, GPT-5’s use of `NoProposal` as a model value (rather than a tuple alias) combined with the terse constraint-heavy style produced invalid TLA+. The strongest result is GPT-5 + `PromptX`, at 100% TV and 4/4 invariants.

3.5 Experiment 3: Configuration Scaling

Experiment 3 evaluates both generated specifications under three TLC configurations that scale the number of participants and the ballot bound. Since both models only succeeded with their own refined prompts, we evaluate both final specifications rather than selecting a single prompt/model combination. Both specs are checked under identical configurations with a strict 90-second cap. This cap makes the two specifications directly comparable regardless of their final state-space size. Configurations ran can be found in Table 4 while results can be seen in Table 5. Three findings stand out.

Firstly, the Claude specification is substantially cheaper to verify. It completes both Small and Baseline within the time limit, whereas the GPT-5 specification completes only Small.

Secondly, the GPT-5 specification explores a dramatically larger state space despite identical `.cfg` parameters, reaching roughly 1.86M states at Small versus 1,217 for Claude, and more than 3.7M states at Baseline before timing out. This is caused primarily by the GPT-5 specification’s `HandlePromise` action, which includes an unconstrained stutter branch (`UNCHANGED vars`) representing message loss. For every `Promise` message, TLC must therefore explore both a processed and ignored execution path, causing exponential branching. These scaling trends are consistent with SysMoBench’s findings on `etcd Raft` and `raftkvs` [5].

Finally, `TypeOK` holds in all completed runs for both specifications, confirming type safety at every evaluated scale. The TV score (78.7% for Claude, 100% for GPT-5) remains unchanged across configurations because transition validation is evaluated against fixed execution-trace windows rather than TLC’s explored state space.

4 Limitations

This section discusses limitations of our approach, experimental design, and the conclusions the results can support, beyond the implementation level caveats already noted in Section 2.3.

Scope of the model. Our specification captures only single-decree Essential Paxos: three roles, two phases, and simple majority quorums. Multi-decree consensus, leader election, durable storage, and the negative-acknowledgement and multi-instance mechanisms of the library’s other modules are out of scope. Conclusions about how well LLMs recover a specification therefore apply to a minimal protocol core, not to the full Paxos family or to industrial consensus systems with larger state and richer message sets.

Bounded verification. TLC verifies invariants only within finite configurations. A passing run is a proof over the checked bound (e.g. three acceptors, two proposers, `MaxBallot=3`), not a general proof of correctness for arbitrary participant counts. Experiment 3 shows that even modest scaling exhausts the state space within our time cap, so the configurations we can exhaustively check are inherently limited and larger bounds remain unverified.

Generality of the prompt-refinement findings. Each refined prompt was derived from the specific SANY/TLC errors encountered in our environment (OpenJDK 23.0.1, TLC 1.8.0, fixed model configurations), so the final prompts are tuned to those errors and to the model that produced them, as the non-transfer result in Experiment 2 makes explicit. We therefore claim that iterative refinement outperformed single-pass prompting *in this setting*, not that any particular prompt constitutes a portable recipe across models, protocols, or toolchain versions.

Experimental power. The study compares two models on one protocol across a small number of refinement runs. With a single subject system and no repeated trials per configuration, we report observed trajectories rather than statistically estimated effects. The iteration counts (8 vs. 4) are illustrative of the refinement process, not a calibrated measure of relative model efficiency.

Table 3: SysMoBench results across (model, prompt) cells

Model	Prompt	Compile	Runtime	TV rate	Invariants
Claude Sonnet 4.5	Prompt _C	PASS	PASS	78.7% (48/61)	4/4
Claude Sonnet 4.5	Prompt _X	FAIL	—	—	—
GPT-5	Prompt _C	FAIL	—	—	—
GPT-5	Prompt _X	PASS	PASS	100% (61/61)	4/4

5 Related Work

TLA+ verification of consensus protocols. Lamport himself has published TLA+ specifications of the Paxos consensus algorithm [7] and various Paxos variants such as Fast Paxos [8]. These hand-written specifications serve as the gold standard for correctness and are the basis for the safety invariants we use in this study. Howard and Mortier [9] provide a comparative formal analysis of Raft and Paxos consensus protocols in TLA+, demonstrating that even subtle implementation differences lead to distinct state machines. Our work differs in that the specification is *generated*, not hand-written, and our goal is to characterize how well LLMs can recover a reference specification from its Python implementation. Unlike Lamport’s own TLA+ specs [7], which cover the full protocol family, we restrict our model to the `essential.py` core, making the comparison between prompt strategies cleaner by holding protocol complexity constant.

Raft and industrial consensus in TLA+. The Raft consensus protocol [10] ships with an official TLA+ specification that has been model-checked against safety invariants similar to those we use. The SysMoBench benchmark itself [5] includes `etcd` Raft and `Redis` Raft as tasks, providing directly comparable LLM generation results. Our Essential Paxos task is simpler than Raft, making it a more controlled setting for studying prompt-engineering effects and comparing the results with the official specification.

LLM-assisted formal specification. Several recent works explore using LLMs to generate or repair formal specifications. Endres et al. [11] investigate the *nl2postcond* task, in which LLMs translate natural-language program intent into formal postconditions. They report that generated specifications are frequently syntactically correct and often useful for bug detection, but that correctness does not guarantee completeness with respect to developer intent. Our findings exhibit a similar phenomenon in a different setting. Generated TLA+ specifications often compile, model-check, and satisfy structural correctness criteria, yet still diverge from the reference implementation’s behaviour under transition validation. This suggests that semantic alignment remains a central challenge across

both assertion generation and executable formal-specification synthesis. The closest precedent to our work is still the SysMoBench paper itself [5], which evaluates a range of models (GPT-4o, Claude Opus, DeepSeek-R1) on eleven systems using the same four-stage pipeline we adopt. SysMoBench reports that no model achieves a top-tier score on all systems, and that transition validation is the hardest stage for every model evaluated. Our contribution extends this evaluation to Essential Paxos and studies whether iterative prompt refinement, rather than model capability alone, is the decisive factor.

Prompt engineering for code generation. Wei et al. [12] show that chain-of-thought prompting improves multi-step reasoning in LLMs. Subsequent work has applied structured and feedback-driven prompting to program synthesis. Chen et al. [13] propose *self-debugging*, in which a model iteratively revises its generated code in response to execution results, and the broader *self-repair* paradigm it exemplifies has since been studied across models and benchmarks [14]. Our iterative refinement procedure is an instance of this paradigm applied to TLA+ generation, with SANY and TLC replacing the compiler and test suite. Unlike artifact-level self-repair, the agent closes the loop at the *prompt* level: each verification failure is fed back through the `tv-eval` skill and the agent regenerates the specification autonomously, with no human translation of errors into constraints. The recorded prompt and revision history makes every constraint traceable to a specific failure mode, producing the per-iteration analysis in Tables 1 and 2.

Benchmarks for AI-generated specifications. Beyond SysMoBench, OSVBENCH [15] benchmarks LLMs on specification generation for operating system verification, finding that current models can produce compilable specifications but frequently violate subtle semantic constraints, which is a finding consistent with our observation that syntax and runtime correctness are necessary but insufficient for faithful specifications. Music et al. [16] address the related problem of validating execution traces of distributed programs against TLA+ specifications, identifying practical challenges in bridging implementation level events to specification-level state changes that are directly relevant to SysMoBench’s transition-validation stage. The key novelty of SysMoBench, and therefore of our study, is the use of *actual execution traces* from the reference system as the ground truth for transition validation, rather than relying solely on human-written invariants.

Comparison with lecture analysis techniques. Among the software analysis techniques covered in the course, our study uses *model checking* (TLC). Its principal advantage over dynamic techniques such as testing and fuzzing is exhaustiveness: within a bounded configuration TLC explores *every* reachable state, so a passing run

Table 4: Configuration parameters

Configuration	Small	Baseline	Large
Acceptors	3	3	5
Proposers	1	2	3
Learners	1	1	1
MaxBallot	2	3	3
Values	2	2	2

Table 5: TLC model checking results under 90 s cap (both specs)

Config	Claude + Prompt _C			GPT-5 + Prompt _X		
	States	Distinct	Done?	States	Distinct	Done?
Small	1,217	244	Yes (<1 s)	1,857,794	72,764	Yes (~36 s)
Baseline	1,502,755	222,984	Yes (~26 s)	>3,680,568	>397,309	No
Large	>3,696,573	>779,841	No	>2,548,004	>441,963	No

is a proof of the invariant over that configuration rather than evidence from sampled executions—decisive for consensus safety, which must hold universally. The cost is twofold. First, it requires a hand-built abstract model rather than operating on source directly, introducing the model–implementation gap that our transition-validation stage is specifically designed to measure. This gap is avoided through symbolic and concrete execution by reasoning over the actual code paths. Second, exhaustive enumeration suffers state-space explosion, which we observe directly in Experiment 3 and which bounds the configurations we can check; deductive verification (theorem proving) escapes this bound but demands manual proof effort that model checking automates. Our contribution is orthogonal to the choice of technique: we study whether an LLM can *produce* the model that model checking then consumes, shifting the human effort from writing the specification to constraining the prompt.

6 Statement of GenAI Use

This project uses GenAI in two ways, consistent with the course policy:

- (1) **Initial codebase exploration.** LLM was used for initial SysMoBench repository exploration. We asked questions like "How does harness work?", "What files do we need to generate?", "Give us a repo architecture overview".
- (2) **Specification generation.** The TLA+ specification (EssentialPaxos.tla) was generated by Claude Sonnet 4.5 (Anthropic) and Chat GPT 5 (Open AI) via the SysMoBench direct_call pipeline. The specification was *not* hand-edited. All improvements were made exclusively by modifying the prompt.
- (3) **Report drafting and tooling assistance.** Both Claude Code (Anthropic) and Codex (OpenAI) were used to drive the autonomous prompt-refinement loop, execute the transition-validation stage via the tv-eval skill, and assist with harness scripting and report drafting of prompt evaluation pipeline. All technical claims and other report parts were verified and written independently by the authors.

References

- [1] S. Chand, Y. A. Liu, and S. D. Stoller. Formal Verification of Multi-Paxos for Distributed Consensus. CoRR, abs/1606.01387, 2016. <https://arxiv.org/abs/1606.01387>.
- [2] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. Available at: <https://lamport.org/tla/book.html>
- [3] Y. Yu, P. Manolios, and L. Lamport. Model Checking TLA+ Specifications. *Correct Hardware Design and Verification Methods (CHARME'99)*, LNCS 1703, pages 54–66, Springer, 1999. Available at: <https://lamport.org/pubs/yuanyu-model-checking.pdf>
- [4] M. Chen et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021. Available at: <https://arxiv.org/abs/2107.03374>
- [5] Q. Cheng et al. SysMoBench: Evaluating AI on Formally Modeling Complex Real-World Systems. *ICLR*, 2026. <https://arxiv.org/abs/2509.23130>
- [6] Tom Cocagne. Essential Paxos. <https://github.com/cocagne/paxos/blob/master/paxos/essential.py>
- [7] L. Lamport. The Paxos Algorithm or How to Win a Turing Award. Available at: <https://lamport.azurewebsites.net/tla/paxos-algorithm.html>
- [8] L. Lamport. Fast Paxos. *Distributed Computing*, 19(2):79–103, 2006. Available at: <https://doi.org/10.1007/s00446-006-0005-x>
- [9] H. Howard and R. Mortier. Paxos vs Raft: Have we reached consensus on distributed consensus? *Proceedings of the 7th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*, 2020. Available at: <https://doi.org/10.1145/3380787.3393681>
- [10] D. Ongaro. Consensus: Bridging Theory and Practice. PhD thesis, Stanford University, 2014. Available at: <https://ongardie.net/var/blurbs/pubs/dissertation.pdf>
- [11] M. Endres et al. Can LLMs Transform Natural Language Intent into Formal Method Postconditions? *Proceedings of the ACM on Software Engineering (FSE)*, 1(FSE):Article 84, 2024. <https://doi.org/10.1145/3660791>
- [12] J. Wei et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS*, 2022. Available at: <https://arxiv.org/abs/2201.11903>
- [13] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching Large Language Models to Self-Debug. *ICLR*, 2024. arXiv:2304.05128. Available at: <https://arxiv.org/abs/2304.05128>
- [14] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama. Is Self-Repair a Silver Bullet for Code Generation? *ICLR*, 2024. arXiv:2306.09896. Available at: <https://arxiv.org/abs/2306.09896>
- [15] X. Wang et al. OSVBENCH: Benchmarking LLMs on Specification Generation Tasks for Operating System Verification. *AAAI Conference on Artificial Intelligence*, 2026. <https://arxiv.org/abs/2504.20964>
- [16] H. Music, O. Gurov, and L. Lamport. Validating Traces of Distributed Programs Against TLA+ Specifications. *Software Engineering and Formal Methods (SEFM)*, 2024. <https://inria.hal.science/hal-04813639>